

Artificial Intelligence Part B Report

Word Count: 1998

Contents

1. Genetic Algorithm Introduction.....	2
2. Fitness Function	2
3. Experiment Setup	3
3.1. Base Settings	3
3.2. Data collection	3
3.3. Data Processing.....	5
4. Experiment Results	6
4.1. Base	6
4.1. Increased population	10
4.3. Increased gene count	14
4.4. Increased mutation	18
4.5. Stronger motors	22
6. Encoding Scheme Experiment: Evolving Only Motor Genes	26
6.1. Setup	26
6.2. Results.....	27
7. Conclusion	31

1. Genetic Algorithm Introduction

This project incorporates a genetic algorithm into a physics-based arena, to evolve creatures that attempt mountain climbing.

The genetic algorithm entry point, script `'ga_runner_cw_envt.py'`, calls function `'run_genetic_algorithm_experiment'`. For each generation, this function creates a Population instance and runs each creature in the arena simulation through the `'get_fitness_run_creature'` method of the Simulation class. Elite genomes are saved and evolved further, and may be visualized through `'realtime_from_csv_arena'` using PyBullet GUI.

2. Fitness Function

The Simulation class method `'get_fitness_run_creature'` updates creature's motors regularly. It stores properties (e.g. mountain collision steps) used to calculate rewards/penalties during simulation steps and after simulation completion. Rewards and penalties are normalized, scaled and compounded to return fitness.

Rewarded behavior

- Closest horizontal radius to mountain center
- Reduction in horizontal radius to mountain center
- Mountain collisions
- Highest platform reached
- Increase in platform height
- Increase in platform height while colliding with mountain

Penalized behavior

- Increase in horizontal radius from mountain center
- High distance/displacement (horizontal longer paths)
- Useless motor effort (no meaningful movement)
- Flying
- Wall proximity steps
- Wall proximity after simulation completion
- Excessive links

3. Experiment Setup

3.1. Base Settings

Per experiment, a genetic algorithm/genome setting changes, and other settings remain controlled as the base settings (table 1).

Setting	Base value
Population size	30
Gene count	3
Mutation rates	shrink mutation rate: 0.20 grow mutation rate: 0.15 point mutation rate: $0.40 / (1 + 0.03 * \text{generation})$ point mutation amount range: $0.50 / (1 + 0.02 * \text{generation})$
Motor gene scaling	amplitude scaling: 0.95 frequency scaling: 13.00
Genes to be evolved	all

Table 1: Base settings

3.2. Data collection

The 'ga_runner_cw_envt.py' entry point runs the genetic algorithm experiment 3 times with given settings (figure 1).

```
# Running the genetic algorithm 3 times for each experiment setting
if __name__ == "__main__":
    for run in [1, 2, 3]:
        run_genetic_algorithm_experiment(
            generations_count=100,
            pop_size=30,
            gene_count=3,
            experiment_name="base",
            run=run
        )
```

1: Entry point of 'ga_runner_cw_envt.py'

The function 'run_genetic_algorithm_experiment' runs each creature in the simulation through Simulation class 'get_fitness_run_creature'. Along with fitness, this method returns a dictionary containing metrics about overall fitness and specific mountain climbing abilities (figure 2).

```

# metrics dictionary
metrics = {
    "fitness": float(max(0.0, fitness)),
    "min_radius": float(min_radius),
    "long_path_ratio_raw": float(long_path_ratio_raw),
    "highest_platform_height": float(highest_platform_height),
    "mountain_collision_ratio": float(mountain_collision_ratio),
    "flying_ratio": float(flying_ratio),
    "wall_proximity_ratio": float(wall_proximity_ratio),
    "useless_motors_effort_total": float(useless_motors_effort_total),
    "total_motor_effort": float(total_motor_effort),
    "links": float(len(cr.get_expanded_links())),
}

# removing the creature urdf
os.remove(xml_file)

# returning fitness and metrics
return float(max(0.0, fitness)), metrics

```

2: *'Simulation.get_fitness_run_creature' metrics return*

The function *'run_genetic_algorithm_experiment'* writes returned metrics (population elite and mean) per generation to a CSV within the experiment run folder (figure 3).

```

elite_metric_values = metrics_list[int(np.argmax(fits))]

# population mean creature metrics
mean_min_radius = float(np.mean([m["min_radius"] for m in metrics_list]))
mean_highest_platform = float(np.mean([m["highest_platform_height"] for m in metrics_list]))
mean_mountain_contact = float(np.mean([m["mountain_collision_ratio"] for m in metrics_list]))
mean_flying = float(np.mean([m["flying_ratio"] for m in metrics_list]))
mean_wall_proximity_ratio = float(np.mean([m["wall_proximity_ratio"] for m in metrics_list]))
mean_long_path_ratio = float(np.mean([m["long_path_ratio_raw"] for m in metrics_list]))
mean_useless_effort = float(np.mean([m["useless_motors_effort_total"] for m in metrics_list]))
mean_total_motor_effort = float(np.mean([m["total_motor_effort"] for m in metrics_list]))
mean_links = float(np.mean([m["links"] for m in metrics_list]))

# Writing fitness statistics of current generation
# as a new row in the csv file
with open(metrics_path, "a", newline="") as f:
    w = csv.writer(f)
    w.writerow([
        generation,

        # elite creature metrics
        float(np.max(fits)),
        float(np.mean(fits)),
        elite_metric_values["min_radius"],
        elite_metric_values["highest_platform_height"],
        elite_metric_values["mountain_collision_ratio"],
        elite_metric_values["flying_ratio"],
        elite_metric_values["wall_proximity_ratio"],
        elite_metric_values["long_path_ratio_raw"],
        elite_metric_values["total_motor_effort"],
        elite_metric_values["useless_motors_effort_total"],
        elite_metric_values["links"],

        # mean creature metrics
        mean_min_radius,
        mean_highest_platform,
        mean_mountain_contact,
        mean_flying,
        mean_wall_proximity_ratio,
        mean_long_path_ratio,
        mean_total_motor_effort,
        mean_useless_effort,
        mean_links,
    ])

```

3: *'run_genetic_algorithm_experiment' logging metrics*

3.3. Data Processing

Running script `'graph_experiment_results.py'` with a metrics CSV aggregates metrics per generation over experiment runs in the following graph categories.

Graph category	Sub-plot	
	y-axis	x-axis
Fitness	Fitness score	Generation
Horizontal movement	Minimum radius	
	Long path ratio (distance/displacement)	
Mountain climbing	Highest platform	
	Mountain collision steps ratio	
Stability	Flying and wall proximity ratios	
	Total and useless motor effort	
	Links	

Table 2: Graph categories

Graphs visualize comparison between experiment settings in fitness and mountain climbing capabilities. Points represent mean values aggregated over experiment runs and colored regions represent standard deviation. Running script `'table_mountain_radius.py'` generates statistics about elite minimum horizontal radius from mountain center per experiment (final generation and improvement over generations), as that is a key indicator of mountain climbing.

```
# Iterating over each experiment run folder within the experiment folder
for run_folder in run_folders:
    # storing the csv metrics as a data frame
    df = pd.read_csv(os.path.join(run_folder, "metrics.csv"))

    # Storing generations as an array as it will be used as the x axis
    # to calculate area under the curve for generalized metrics
    generations = df["generation"].to_numpy(dtype=float)

    # The elite minimum horizontal mountain radius reached last generation
    # values across are taken from the last row of the dataframe
    final_elite_radius.append(float(df.iloc[-1]["elite_min_radius"]))

    # Storing the elite creature's minimum radius
    # from the mountain across generations as a dataframe
    elite_min_radius_generations = df["elite_min_radius"].to_numpy(dtype=float)

    # Subtracting the minimum radius of elite creatures in subsequent generations
    # from the minimum radius of the first generation elite gives improvement values
    # in minimum radius over generations
    elite_min_radius_improvement = elite_min_radius_generations[0] - elite_min_radius_generations

    # Using area under the curve to integrate the minimum radius improvement
    # values over generations and get a single value to represent improvement
    elite_min_radius_improvement_over_generations = float(np.trapezoid(elite_min_radius_improvement, generations))

    # Dividing improvement in minimum radius over the number of generations
    # to get mean improvement in minimum radius over generations
    improvement_elite_radius.append(elite_min_radius_improvement_over_generations / float(generations[-1]))

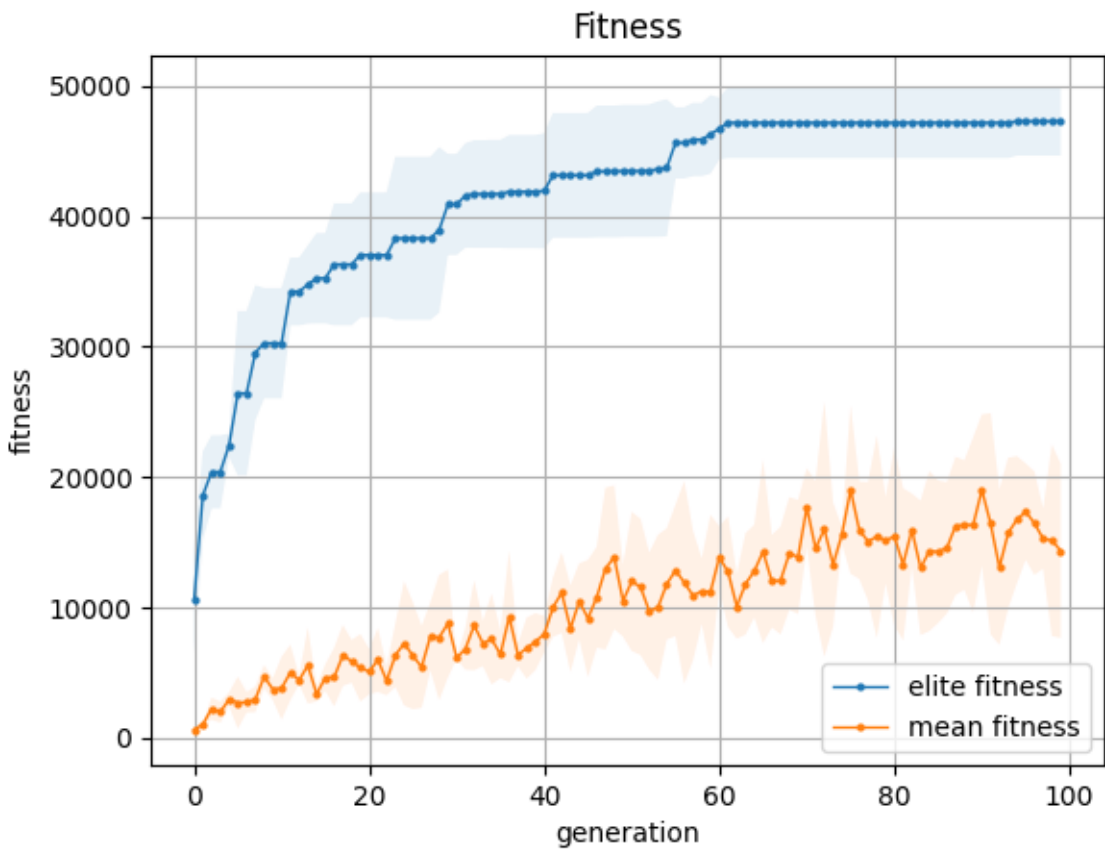
    # storing the experiment name
    experiment_name = os.path.basename(os.path.normpath(experiment_folder))
```

4: Statistics calculation `'table_mountain_radius.py'`

4. Experiment Results

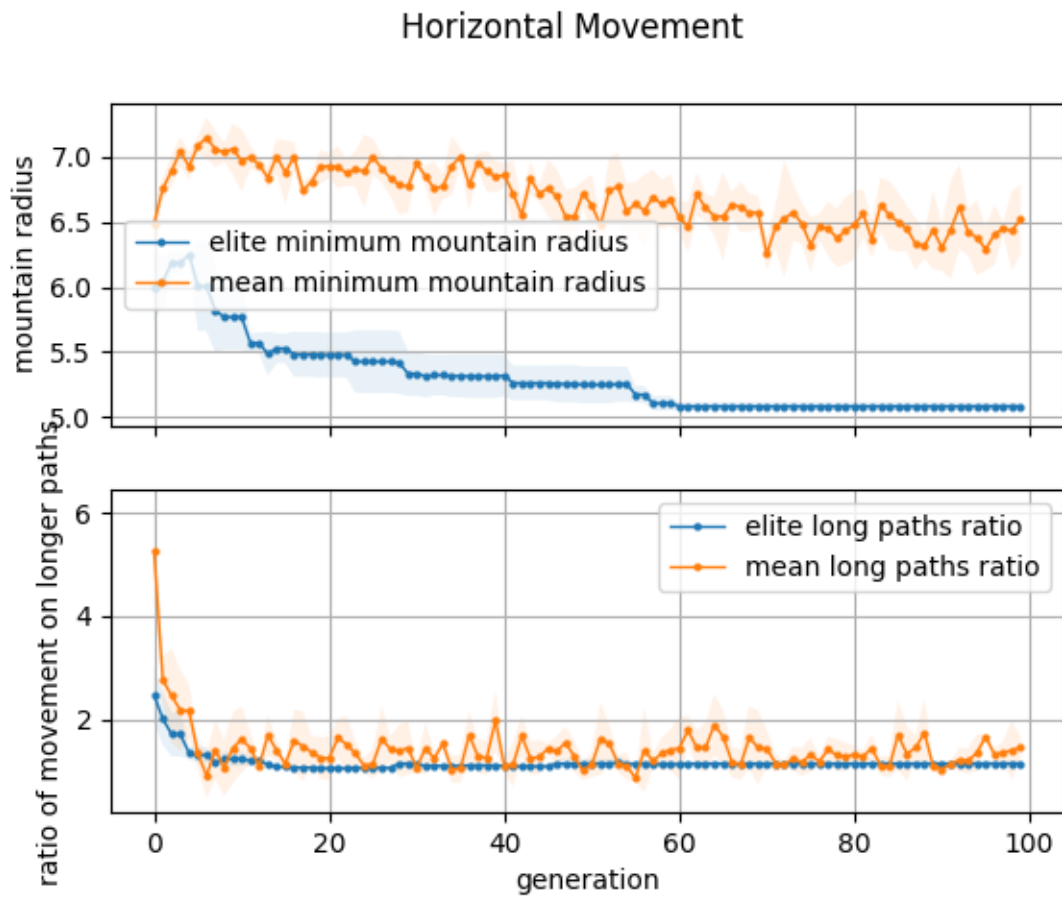
4.1. Base

Figure 5 shows elite fitness increasing with decreasing differentiation, reflecting exploratory genomes earlier and exploitation of mountain climbing patterns later (point mutation rate/range scaling inversely with generations). The large gap between elite and mean fitness reflects that strong mountain climbers constitute a population minority.



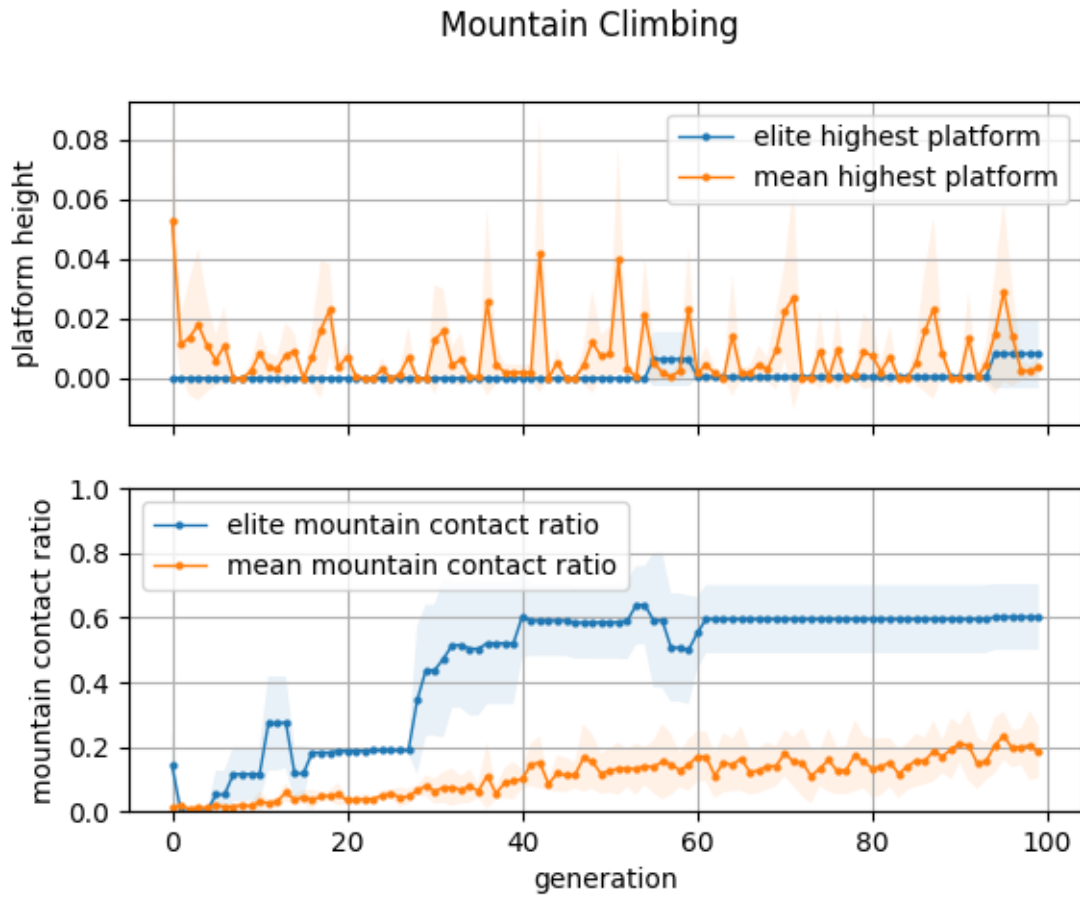
5: Fitness (base settings)

Figure 6 confirms elites gradually learn horizontal mountain approach, but learning converges at 5.0 radius. The elite distance/displacement (long path ratio) converges early to almost 1, suggesting that elites quickly learn optimal paths.



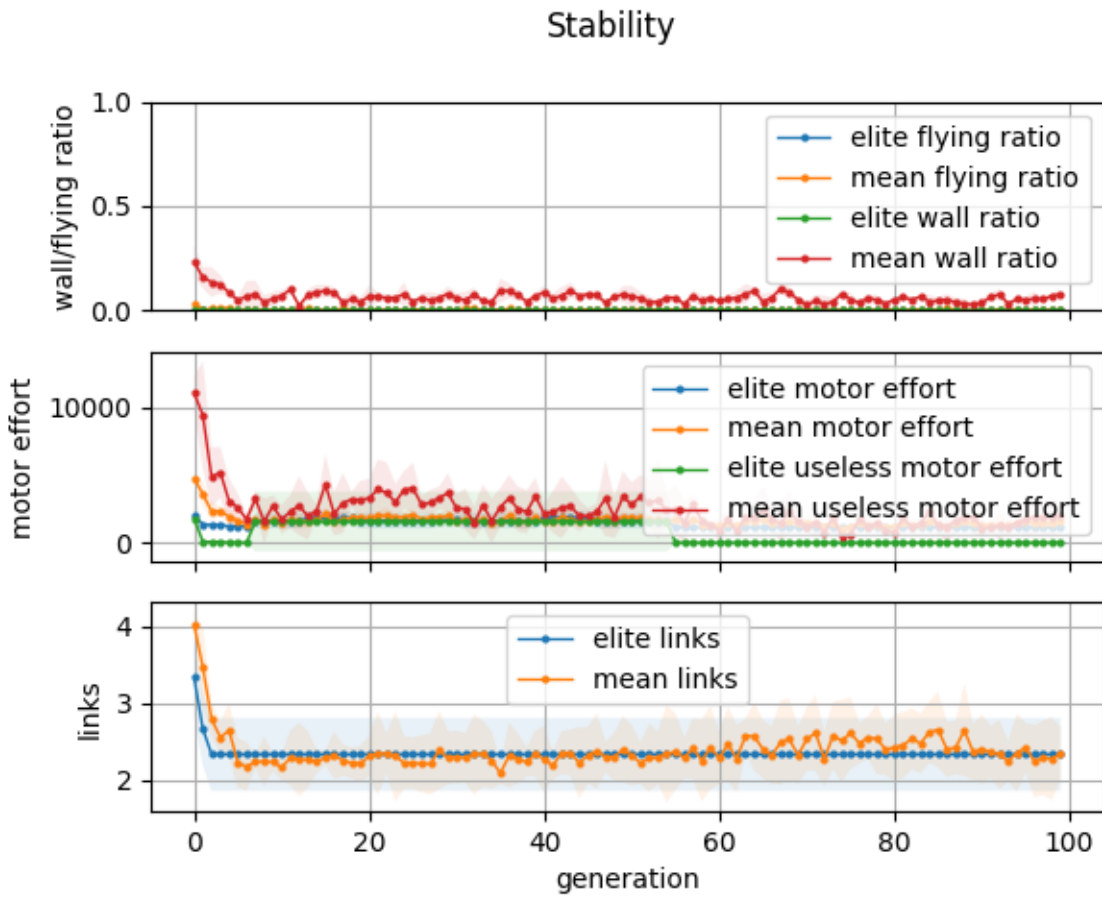
6: Horizontal movement (base settings)

Figure 7 reveals elites gradually learning to maintain mountain contact, converging to 60% of simulation. Highest platform reached does not show notable increase, indicating that elites reach the mountain but do not climb it. Mean creatures show platform height hikes, but the high mean radius suggests these hikes do not equate to mountain climbing.



7: Mountain climbing (base settings)

Figure 8 shows positive elite stability as links converge between 2 and 3. Behaviors that hinder mountain climbing (flying, wall proximity, useless motor effort) are not significant.

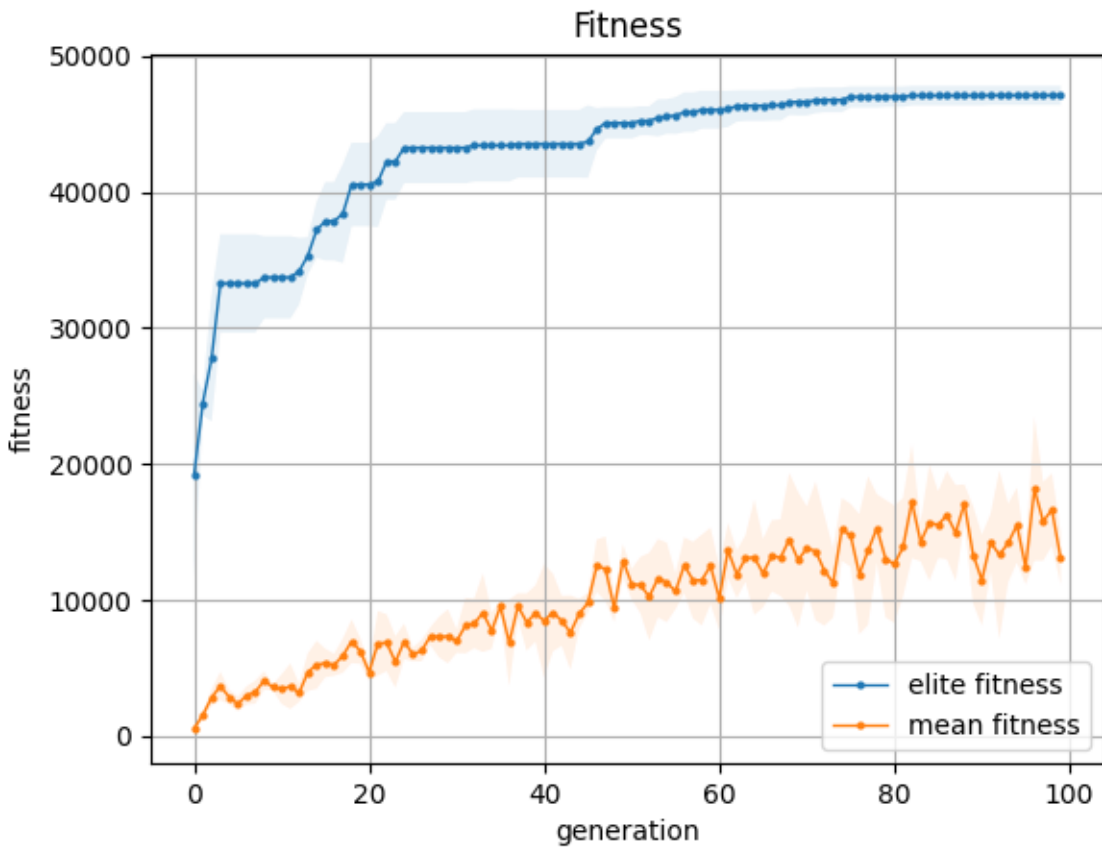


8: Stability (base settings)

4.1. Increased population

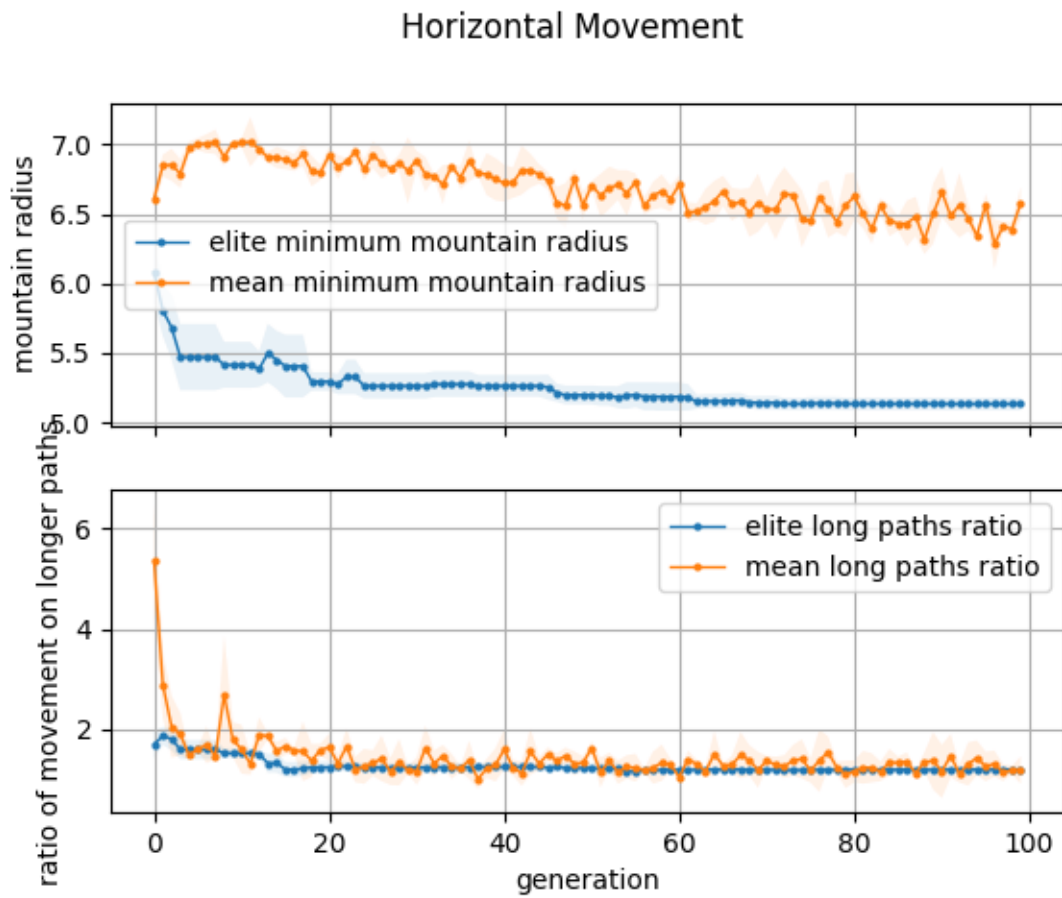
To unlock wider solutions, ‘*run_genetic_algorithm_experiment*’ function ‘*pop_size*’ input was increased to 50 (base settings controlled).

Figure 9 shows elite fitness hiking earlier, with less differentiation, narrower standard deviation and not becoming completely flat like base settings. This suggests population diversity discovered the best solution earlier, reliably and allowed minor refinement later.



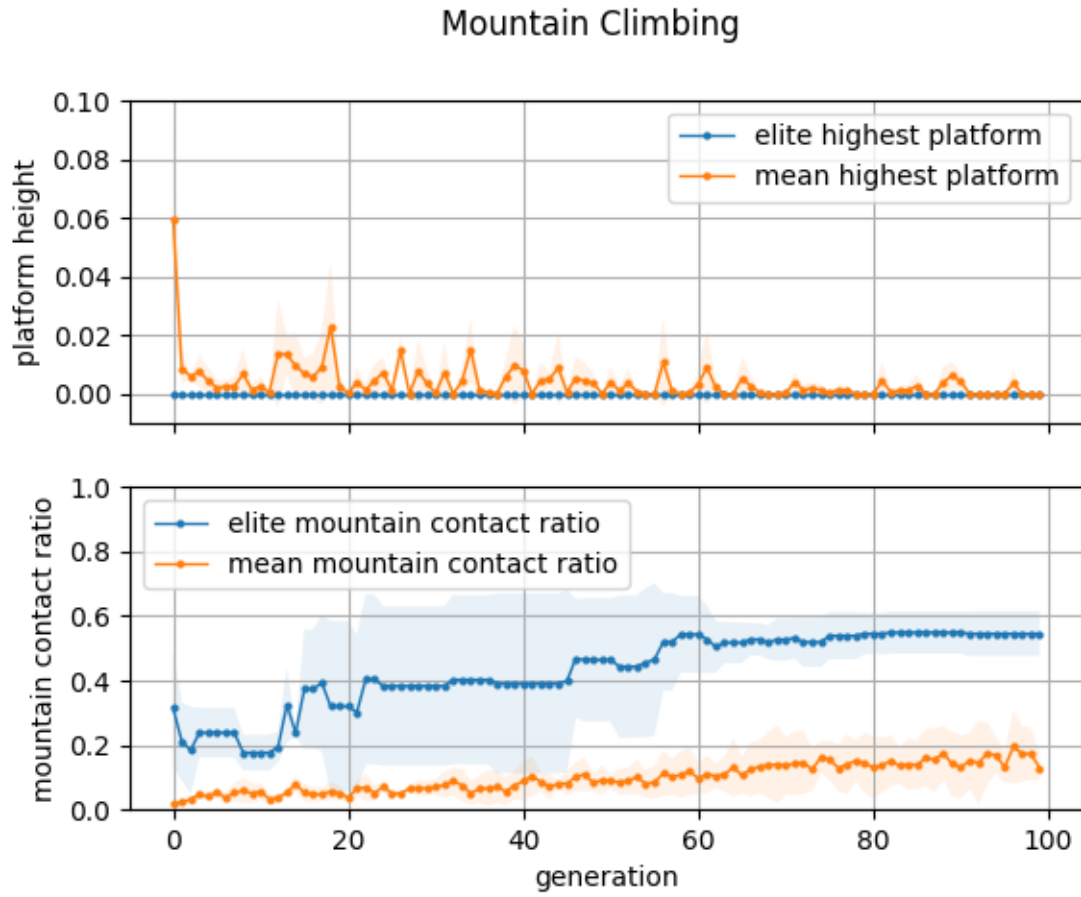
9: Fitness (increased population)

Figure 10 shows that increased population causes earlier reduction in elite mountain radius, converging to similar radius as the base experiment. The mean long path ratio does not hike as much, as inefficient movement is less dominant in a diverse population.



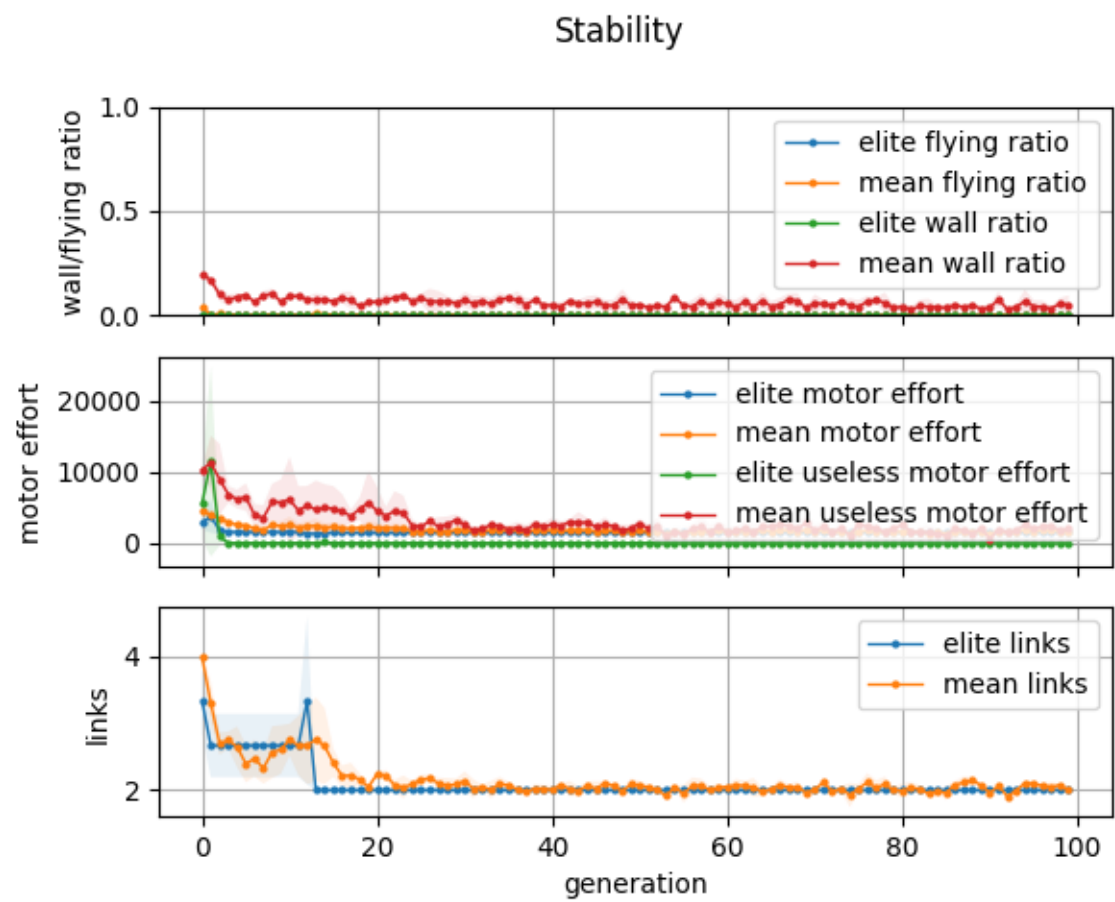
10: Horizontal movement (increased population)

Figure 11 shows elite mountain contact converging to a similar value as the base experiment with narrower standard deviation. Elite highest platform height does not increase. Hence, increased population created reliable mountain contact without unlocking ascent.



11: Mountain climbing (increased population)

Figure 12 shows minimal flying, wall proximity and useless motor efforts, so increased population promotes diversity without compromising stability. Links converge later, suggesting increased population facilitates morphological exploration.



12: Stability (increased population)

Figure 13 shows elite minimum radius from the mountain for increased population reaches higher than base settings in the last generation but shows greater improvement over generations.

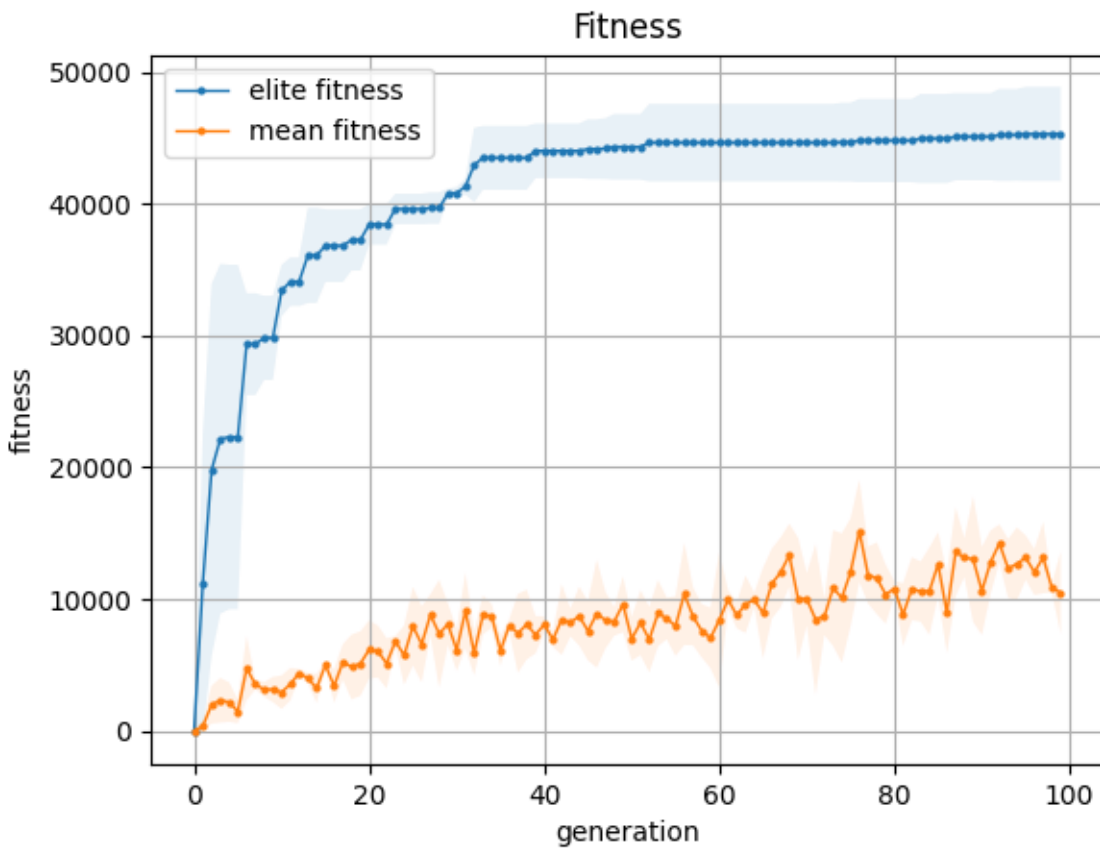
Experiment	Last generation elite minimum radius	Improvement in elite minimum radius over generations
base	5.081, std: 0.028	0.690, std: 0.277
population_50	5.133, std: 0.044	0.835, std: 0.188

13: Mountain radius

4.3. Increased gene count

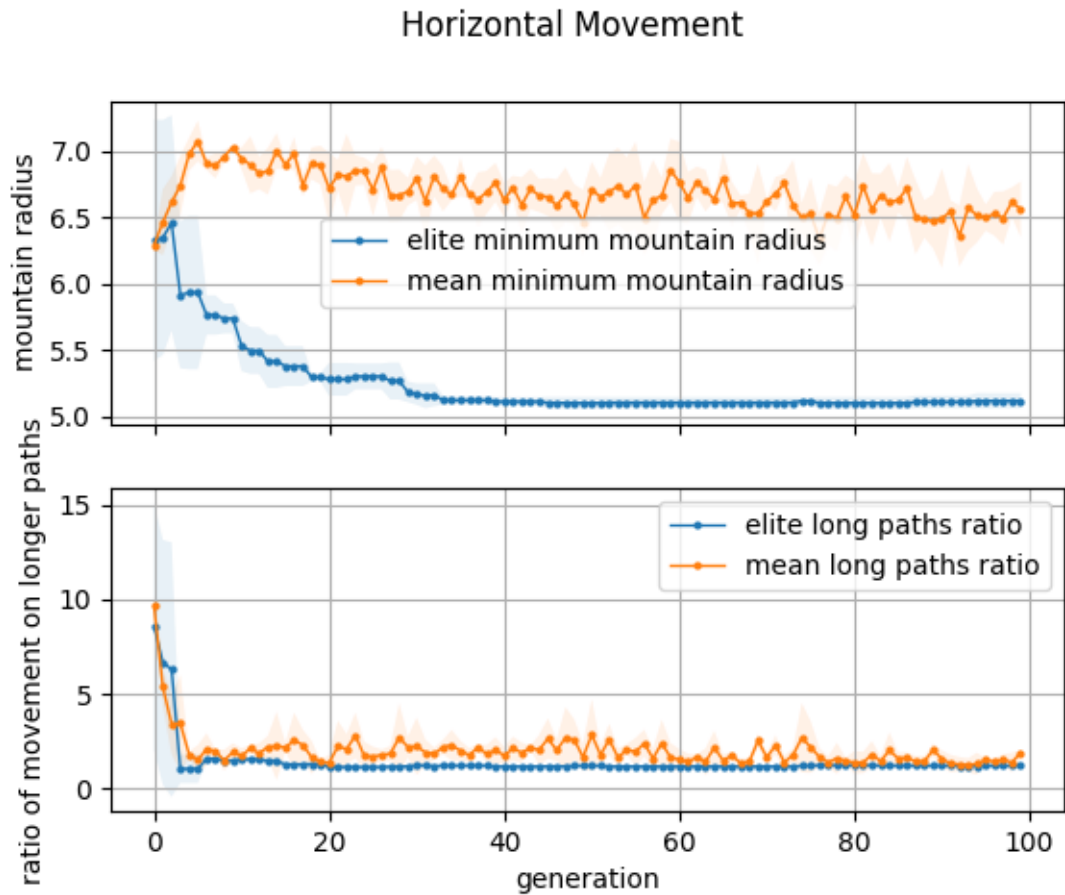
To unlock different solutions, ‘*run_genetic_algorithm_experiment*’ function ‘*gene_count*’ input was increased to 5 (base settings remained controlled).

Figure 14 shows the elite/mean fitness reach lower than the base experiment, with elite fitness converging early. Increasing gene count compounded search complexity, leading to early settlement on inadequate genomes.



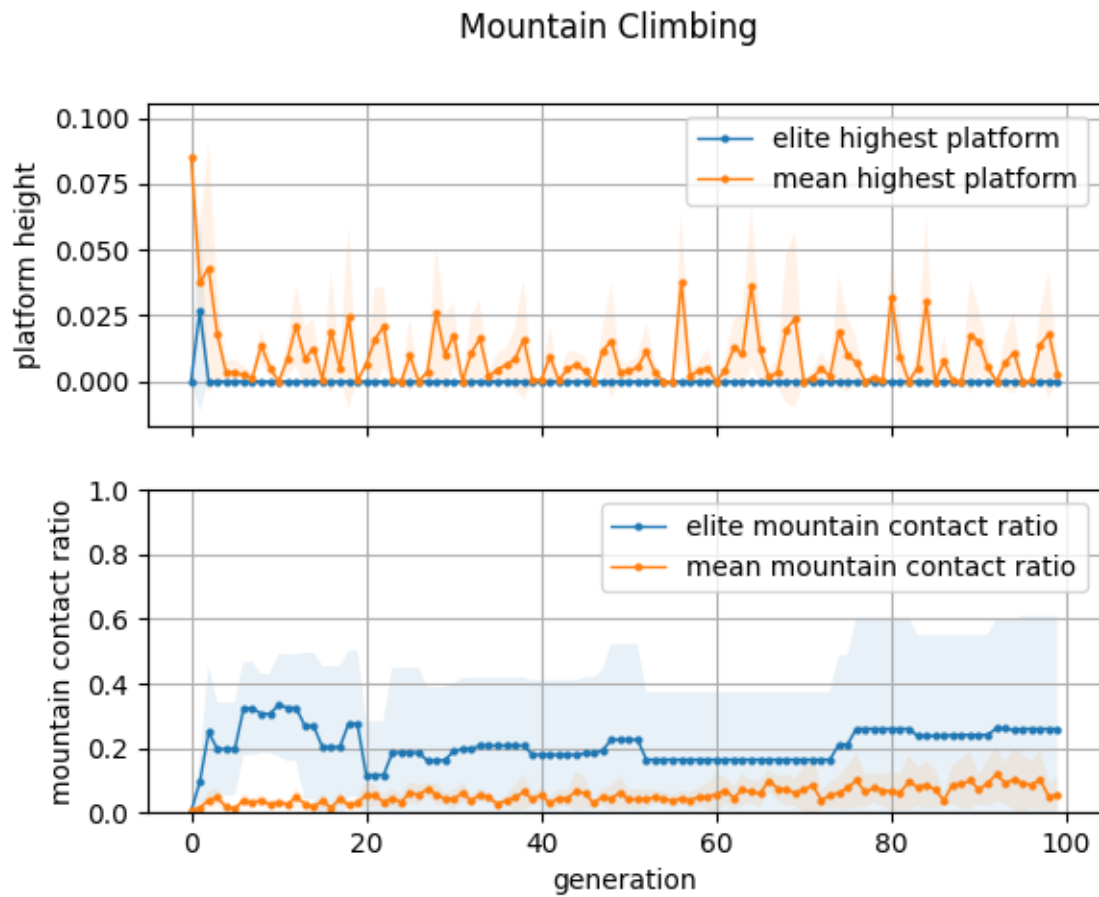
14: Fitness (increased genes)

Figure 15 shows that elite/mean minimum radius is higher than base settings, with elite values converging earlier. However, the distance/displacement (long path ratio) is almost 1 (similar to base settings). Early convergence on locally optimal solutions makes creatures stop before the mountain.



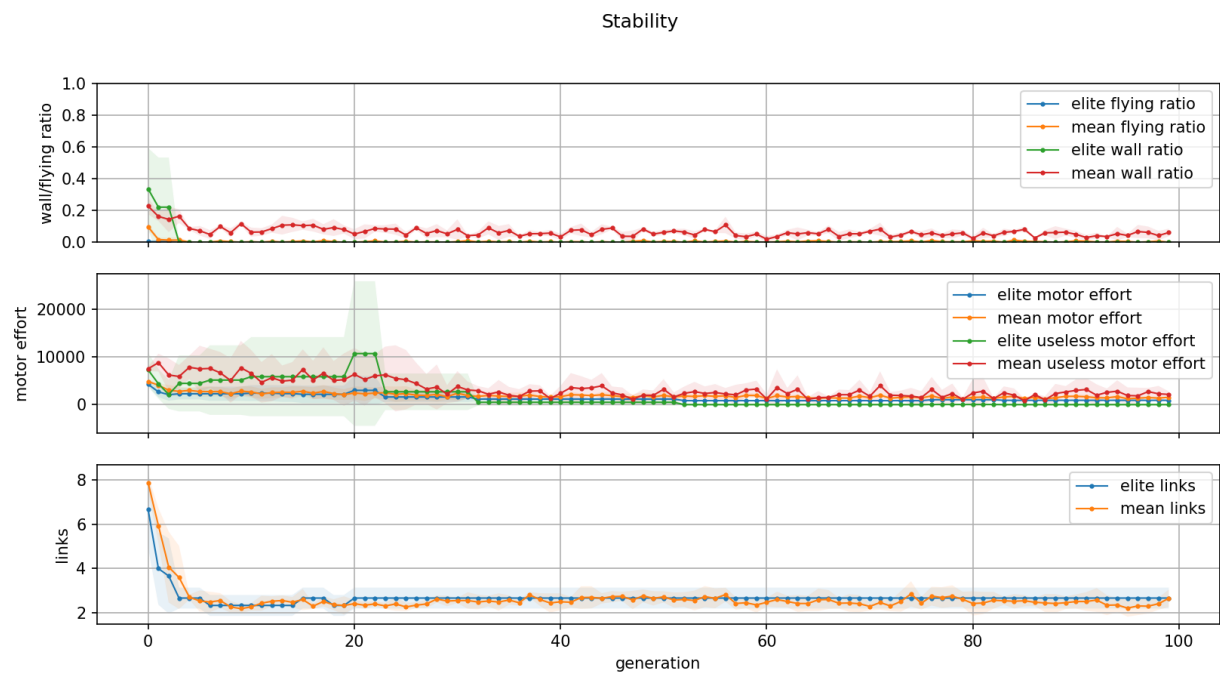
15: Horizontal movement (increased genes)

Figure 16 shows that elite/mean mountain contact ratio drops from base settings (elite dropped to almost half) and highest platform height remains low. This aligns with creatures settling before reaching the mountain.



16: Mountain climbing (increased genes)

Figure 17 shows that wall proximity ratio, flying ratio and links remain similar to base settings. Elite/mean useless motor effort hike up significantly with wide standard deviation. Increasing genes preserves structural stability but causes motor instability.



17: Stability (increased genes)

Figure 18 shows elite minimum radius from the mountain center with increased genes reaches higher in the last generation but shows significantly greater improvement over all generations. Increased genes facilitate wide exploration but cause poor solution refinement.

Experiment	Last generation elite minimum radius	Improvement in elite minimum radius over generations
base	5.081, std: 0.028	0.690, std: 0.277
gene_5	5.118, std: 0.057	1.090, std: 0.913

18: Mountain radius

4.4. Increased mutation

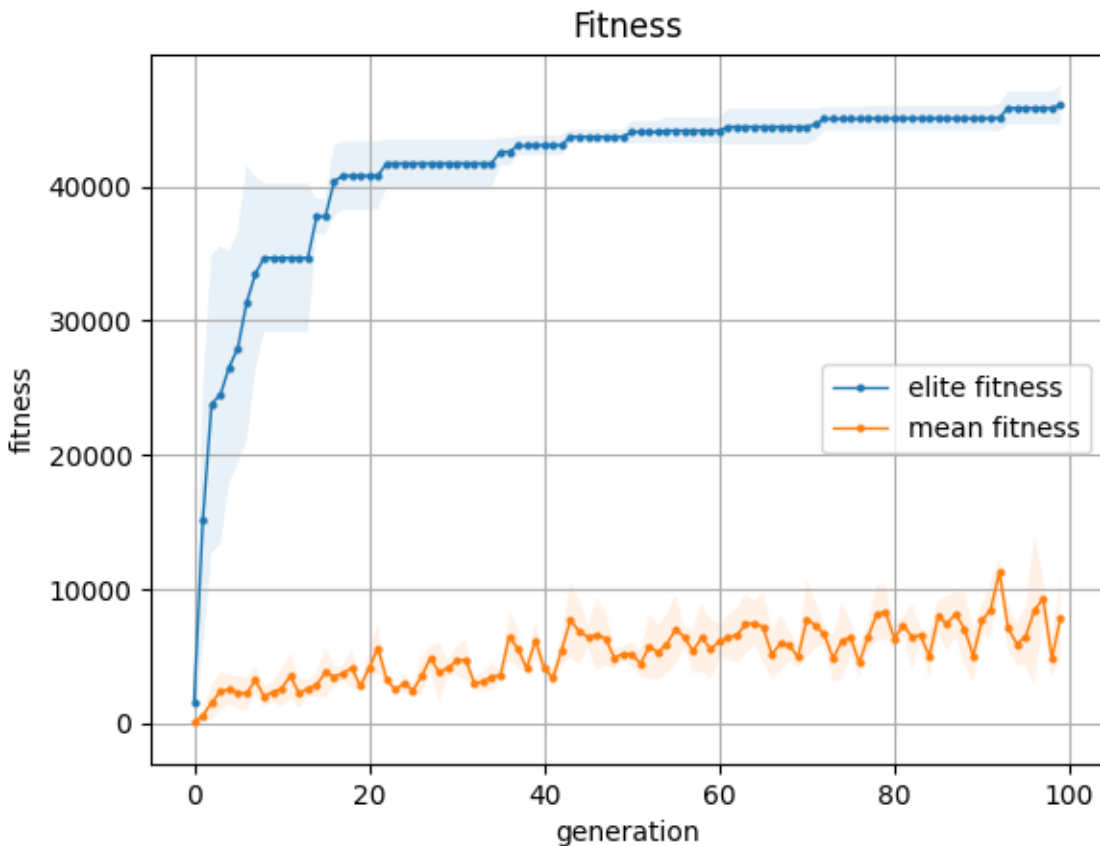
To check if increased mutation escapes locally optimal solutions, mutations rates/range were increased from base settings as shown in figure 19 (other settings controlled).

```
# tuning shrink/grow mutation rates
shrink_mutate_rate = 0.30
grow_mutate_rate = 0.30

# Making point mutation rate and amount range scale inversely
# with generation so mutation decreases in later generations
point_mutate_rate = 0.60 / (1 + 0.03 * generation)
point_mutate_range = 0.70 / (1 + 0.02 * generation)
```

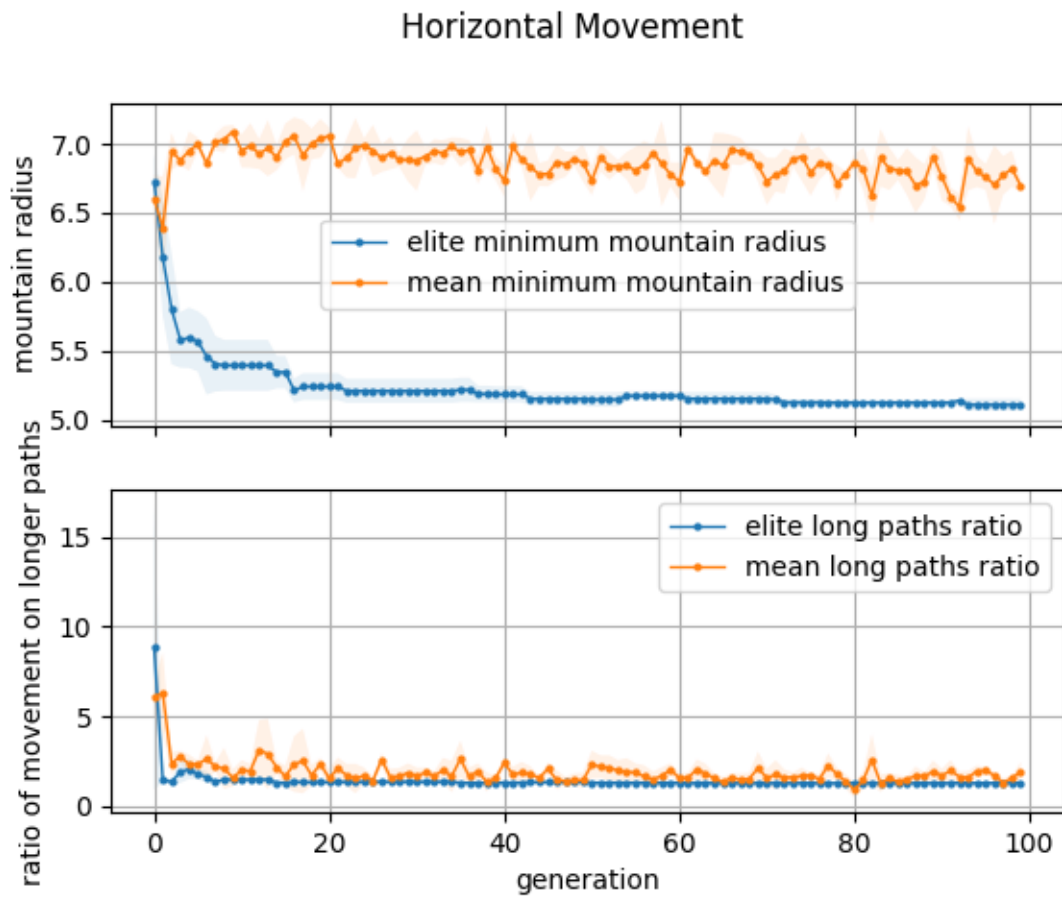
19: Increased mutation

Figure 20 shows elite fitness converging significantly earlier, refining continuously and with narrower standard deviation than the base experiment, but mean fitness is lower. This suggests that increased mutation raises elite fitness faster/reliably but hinders population-wide fitness.



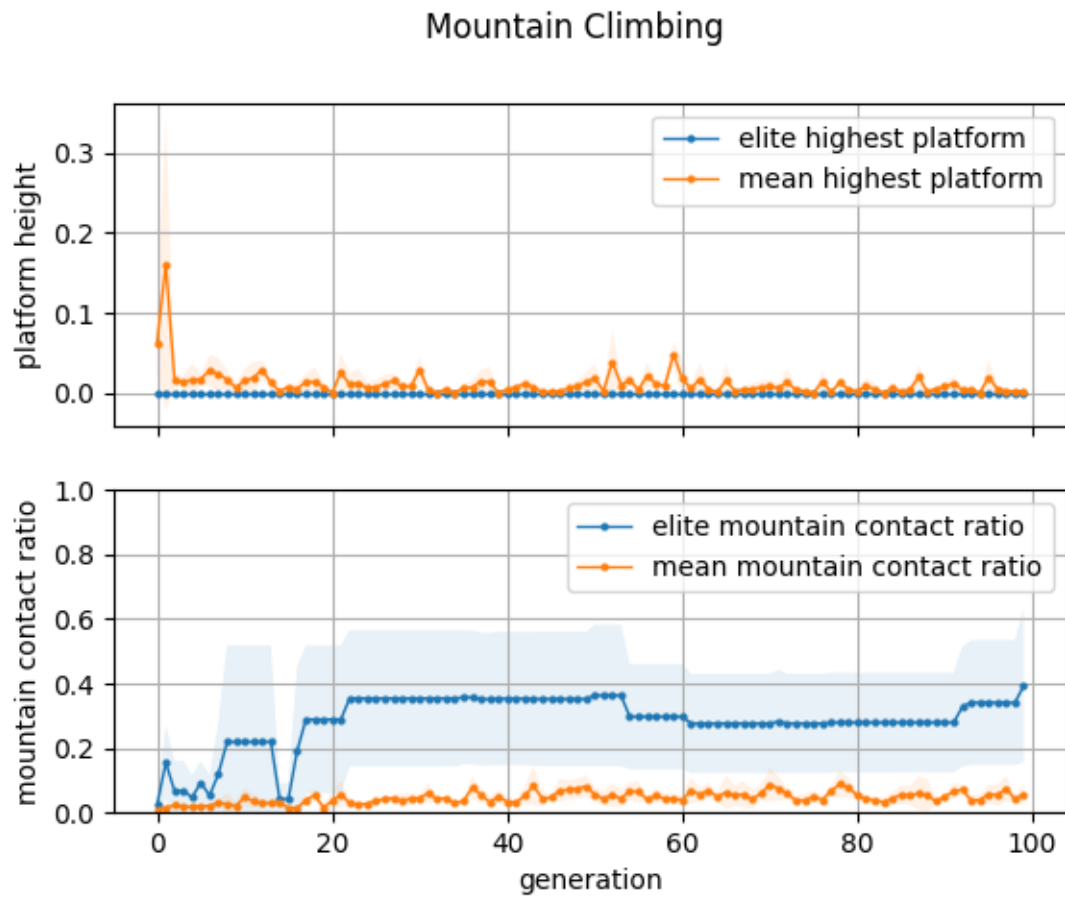
20: Fitness (increased mutation)

Figure 21 shows elite minimum mountain radius converging earlier and to a similar value as base settings, but mean creature radius is higher. This reinforces that increased mutation makes elites learn mountain approach earlier but hinders general population learning.



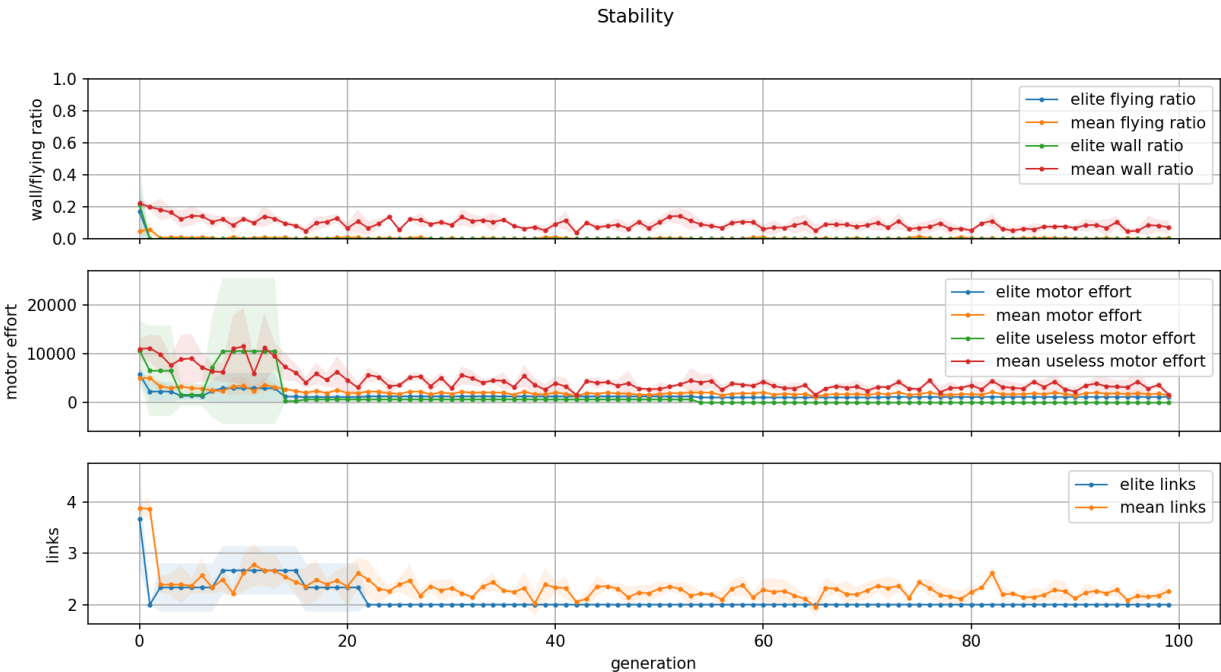
21: Horizontal movement (increased mutation)

Figure 22 shows no increase in elite highest platform from base settings, and elite mountain contact dropped significantly with wide standard deviation. This indicates that high mutation makes creatures learn mountain approach faster but reduces reliable mountain contact/engagement.



22: Mountain climbing (increased mutation)

Figure 23 shows that wall proximity and flying ratios are similar to base settings, useless motor effort is significantly higher in earlier generations, and elite links converge later. Higher mutation initially disturbs motor controls and morphology but settles eventually.



23: Stability (increased mutation)

Figure 24 shows that high mutation results in greatest improvement in elite minimum mountain radius over generations, but final generation value is slightly higher. High mutation facilitates greater exploration early, but compromises on later refinements.

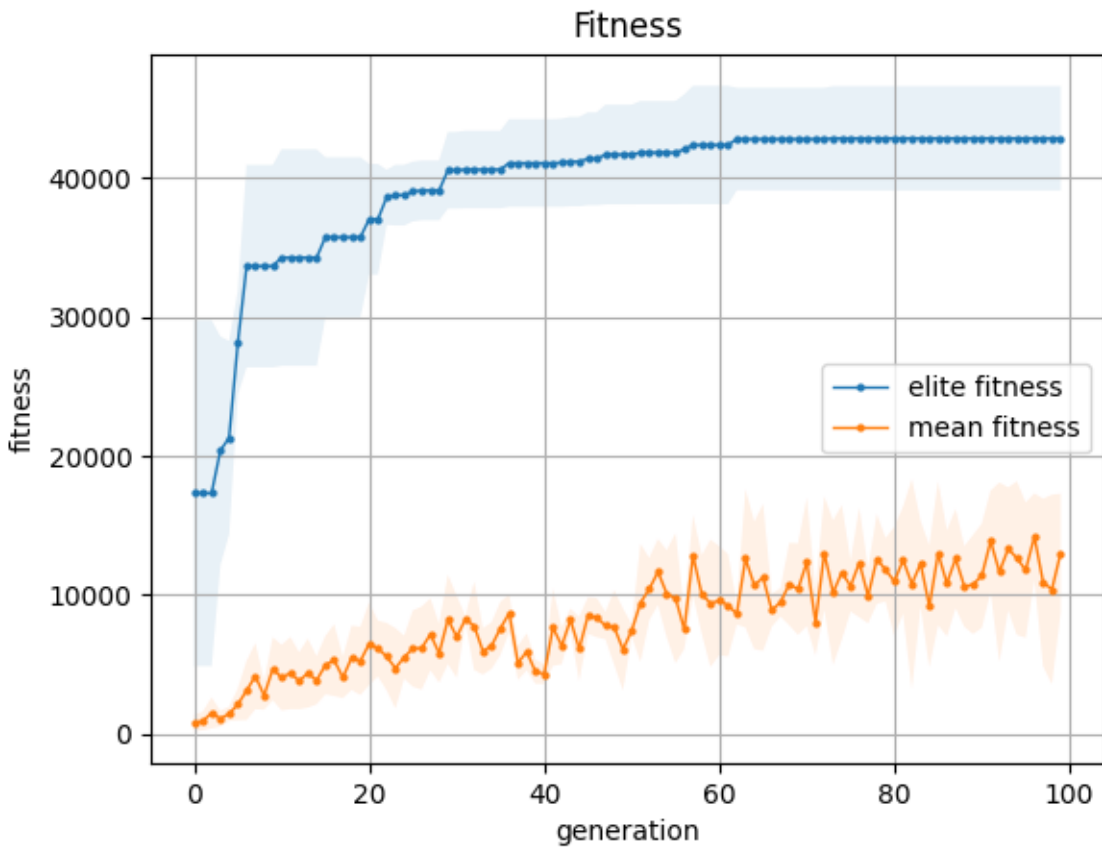
Experiment	Last generation elite minimum radius	Improvement in elite minimum radius over generations
base	5.081, std: 0.028	0.690, std: 0.277
mutation_high	5.109, std: 0.042	1.507, std: 0.190

24: Mountain radius

4.5. Stronger motors

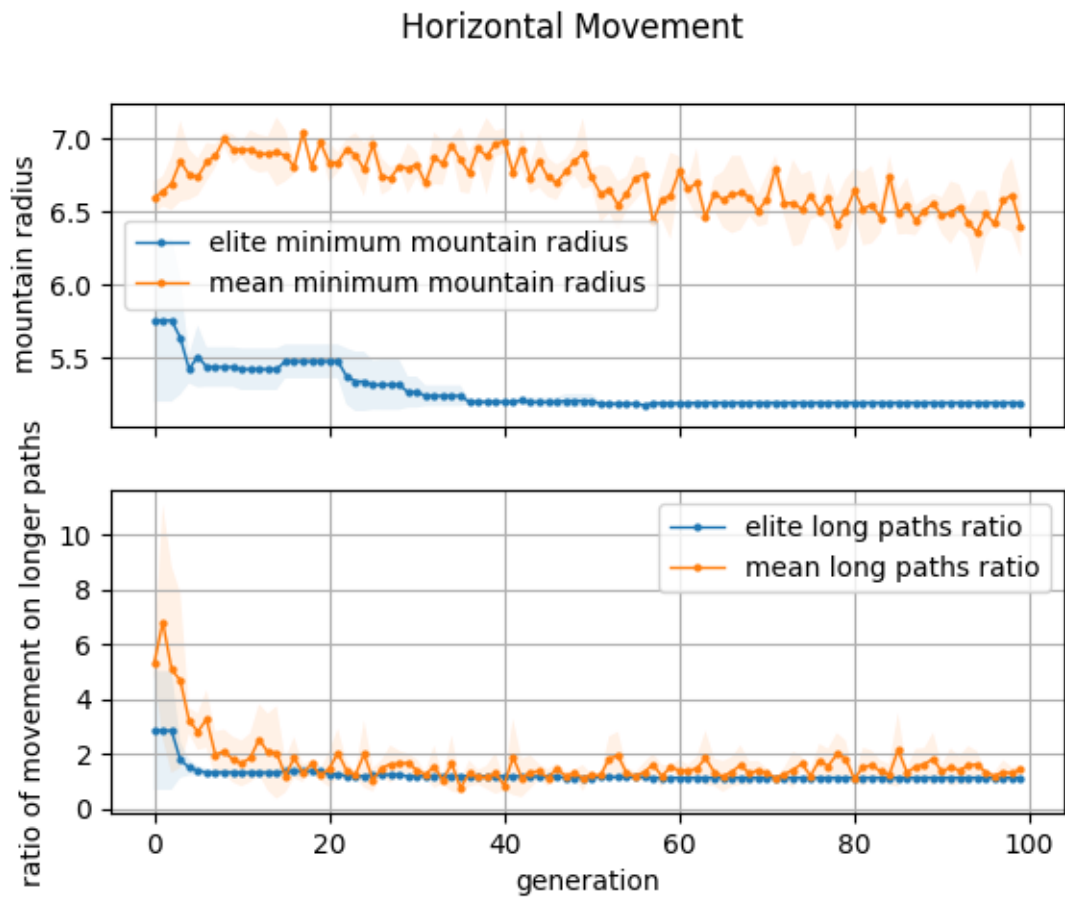
In the `'get_gene_spec'` method of the Genome class, motor amplitude scale was increased to 1 and frequency scale increased to 15, to check if stronger motors unlock climbing (other settings controlled).

Figure 25 shows that elite fitness converges earlier, lower and with greater standard deviation than base settings. Motor strength reduces reliability and causes premature solutions.



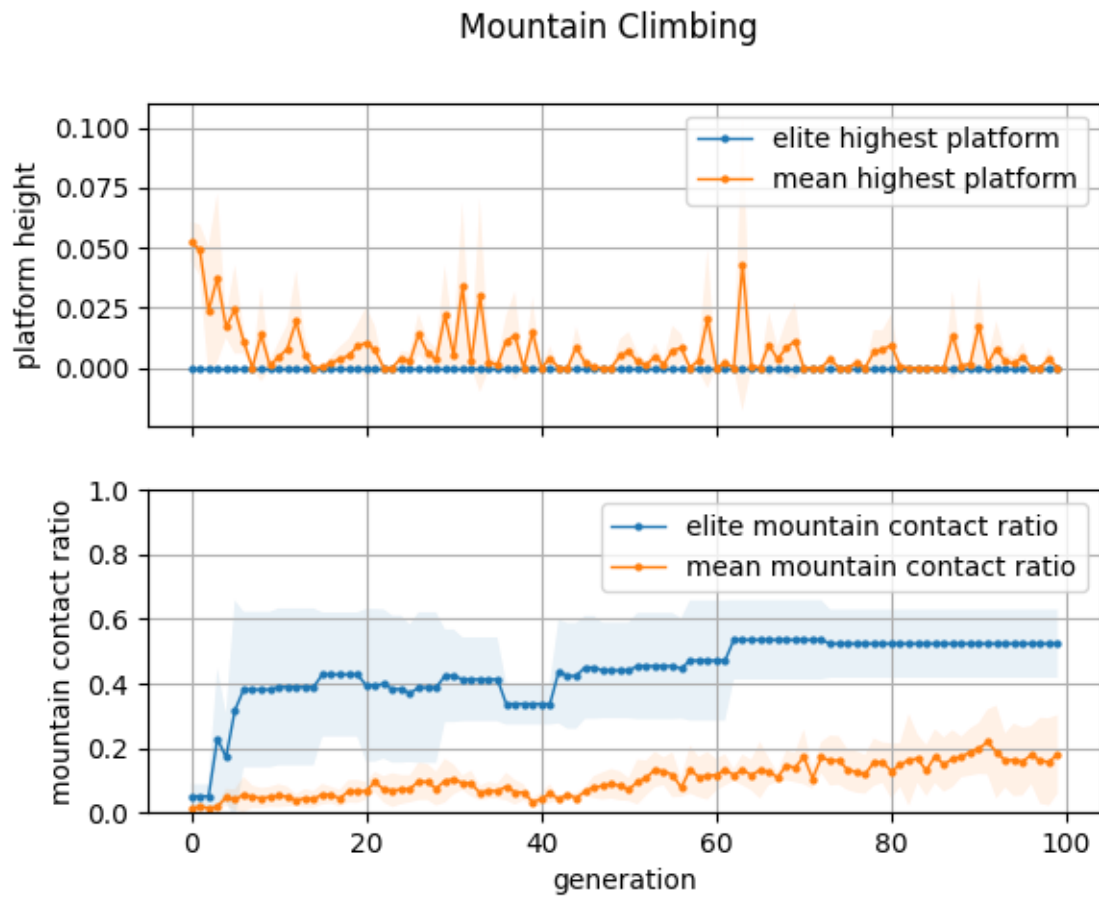
25: Fitness (strong motors)

Figure 26 shows that elite minimum mountain radius converges faster and to a higher value than base settings, reinforcing that motor strength results in premature, unreliable solutions. Elite long path ratio is similar to base settings, because creatures learn to take short paths quickly irrespective of motor strength.



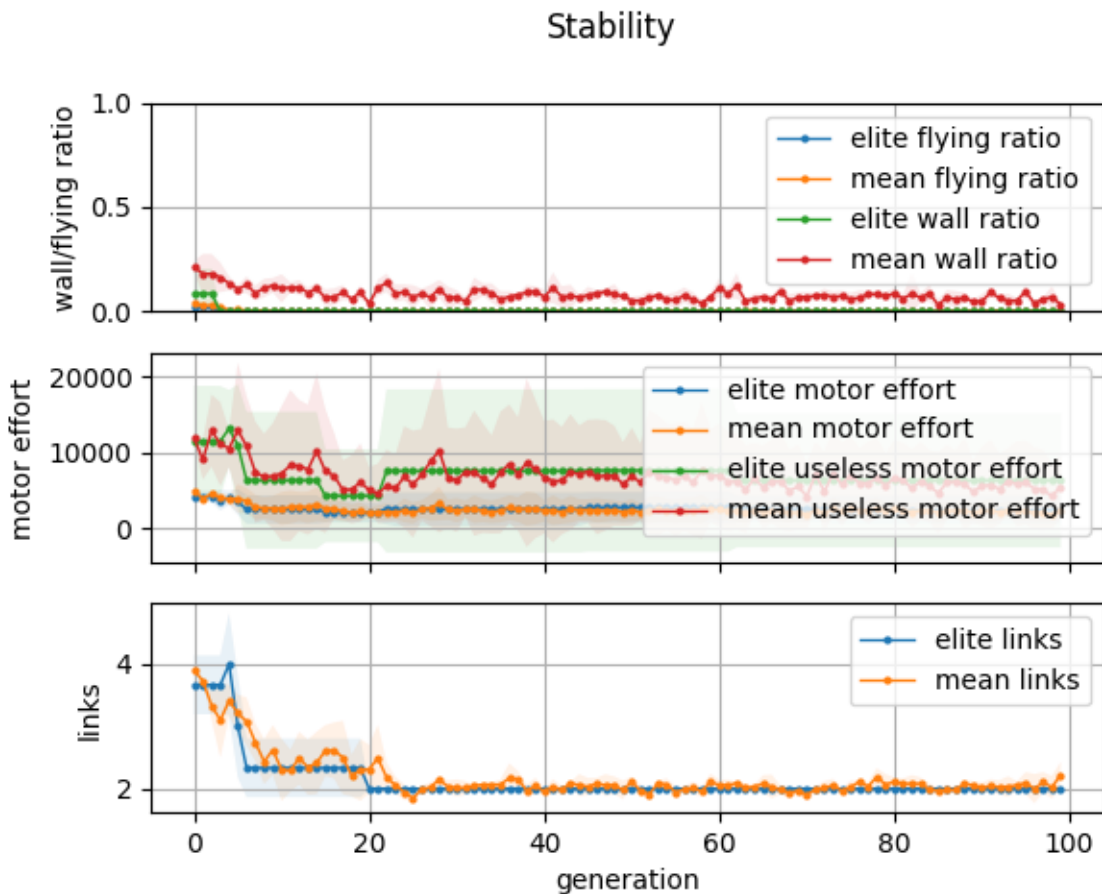
26: Horizontal movement (strong motors)

Figure 27 shows that elite mountain contact ratio converges earlier, lower and with more standard deviation. This reflects that motor strength creates unreliability in motor control and does not translate into mountain climbing.



27: Mountain climbing (strong motors)

Figure 28 shows that motor strength did not create flying or wall approaching instability. However, elite/mean useless motor effort is significantly higher with wide standard deviation. Increasing motor strength does not equate to stronger movement/mountain climbing due to unreliable control.



28: Stability (strong motors)

Figure 29 shows that motor strength produces a higher final generation elite minimum radius, and lesser improvement overall. This reflects lack of reliability of motor control with greater motor strength.

Experiment	Last generation elite minimum radius	Improvement in elite minimum radius over generations
base	5.081, std: 0.028	0.690, std: 0.277
strong_motors	5.195, std: 0.019	0.482, std: 0.484

29: Minimum mountain radius: base settings vs strong motors

6. Encoding Scheme Experiment: Evolving Only Motor Genes

6.1. Setup

This experiment checks if changing the encoding scheme to evolve only motor related genes may increase motor control reliability and unlock climbing (base settings controlled). To implement this, a mode property `'evolve_only_motors'` is created within the `'run_genetic_algorithm_experiment'` function and passed to the Genome class method `'get_gene_indices_to_be_mutated'`. This returns only motor related gene indices (waveform, amplitude, frequency) from the gene spec if the `'evolve_only_motors'` mode is true (figure 30).

```
# This function takes the gene specification from the creature class as input,
# along with the evolve mode (only motors/all genes). It returns the indices of
# genes to be mutated based on the evolve mode
@staticmethod
def get_gene_indices_to_be_mutated(spec, evolve_only_motors):

    # if only motors genes are to be mutated
    if evolve_only_motors == True:
        # returning indices of the three motor related genes
        return [spec[gene]["ind"] for gene in ["control-waveform", "control-amp", "control-freq"]]
    # if all genes are to be mutated
    else:
        # returning all gene indices from the gene specification
        return [spec[gene]["ind"] for gene in spec.keys()]
```

30: `'get_gene_indices_to_be_mutated'`

The `'run_genetic_algorithm_experiment'` function passes the returned indices to the point mutate function, which is altered to skip mutation of unspecified genes (figure 31). Furthermore, shrink/grow mutate is not called in `'evolve_only_motors'` mode.

```
# skipping genes which are not to be mutated
if i not in mutation_gene_indices:
    continue

#####
OF CODE I WROTE MYSELF
#####

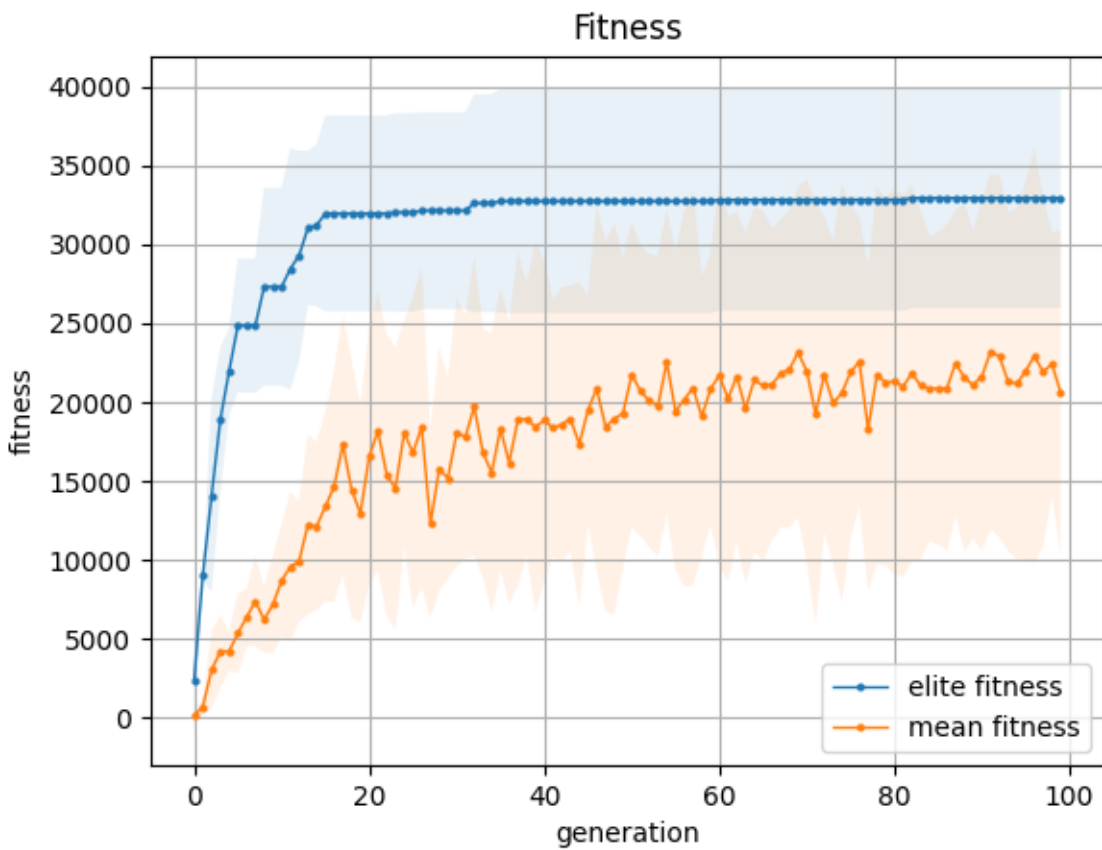
#####
RT OF CODE TAKEN FROM PROVIDED STARTER CODE FILE 'genome.
#####

if random.random() < rate:
    gene[i] += random.uniform(-amount, amount)
if gene[i] >= 1.0:
    gene[i] = 0.9999
if gene[i] < 0.0:
    gene[i] = 0.0
return new_genome
```

31: `'point_mutate'` skipping unspecified genes

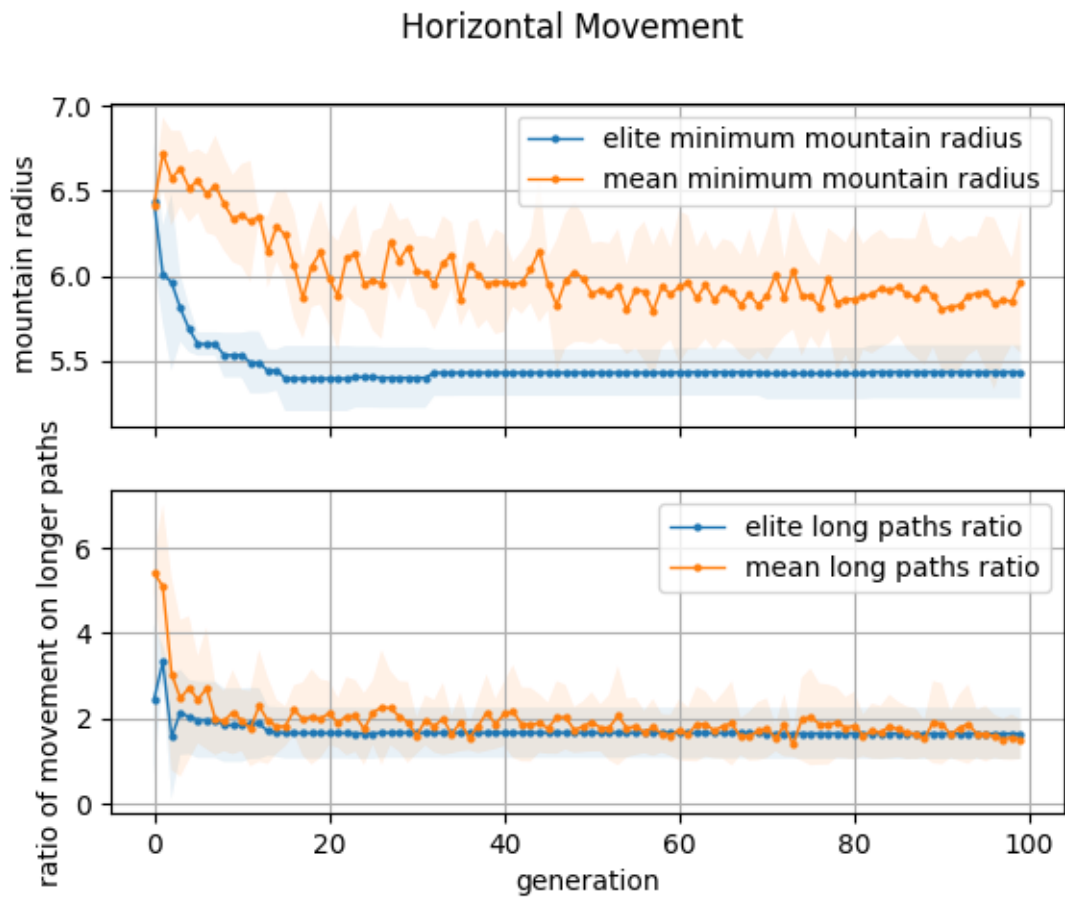
6.2. Results

Figure 32 shows that the elite creature fitness converges earlier, far lower than base settings with greater standard deviation. Restricting morphology limits fitness and reliability.



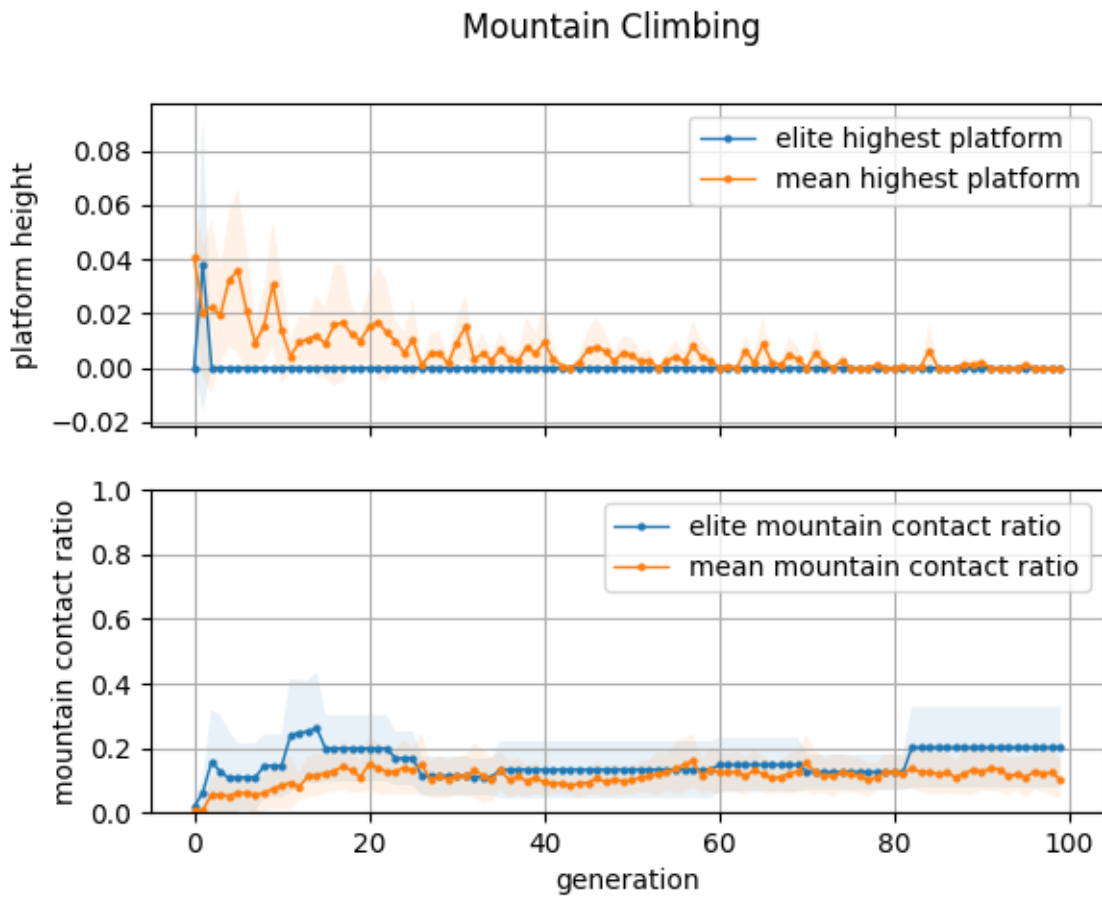
32: Fitness (only evolving motors)

Figure 33 shows that elite minimum mountain radius converges higher than base settings, reinforcing that restricting morphology limits mountain approach. Long path ratio is almost double of base settings, reflecting that paths optimization is linked to morphological evolution, not just motor control genes. Evolving only motors leads to inefficient movement e.g. circling.



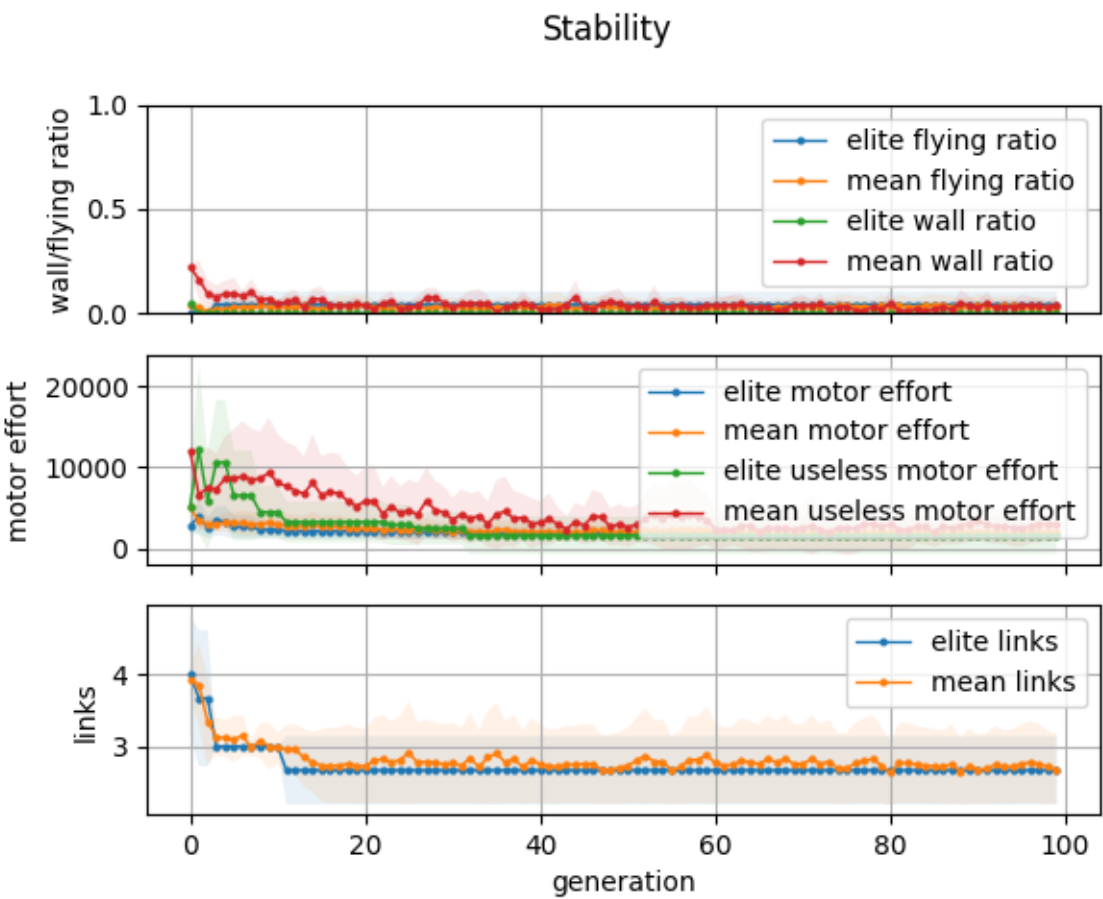
33: Horizontal movement (only evolving motors)

Figure 34 shows mountain contact converges lower than all experiments, and highest platform remains flat. Evolving only motors does not unlock mountain ascent but reduces mountain interaction due to restricted morphological evolution.



34: Mountain climbing (only evolving motors)

Figure 35 shows that evolving motors only does not create wall proximity/flying instability. Elite/mean useless motor effort is significantly higher with greater standard deviation. Inflexible morphological structure hinders translation of motor effort into purposeful/reliable movement.



35: Stability (only evolving motors)

Figure 36 shows that evolving motor related genes produces higher elite minimum radius than base settings in the final generation, but greater improvement over all generations. Wider learning about certain mountain climbing behaviors was facilitated, but inflexible morphology hindered final performance.

Experiment	Last generation elite minimum radius	Improvement in elite minimum radius over generations
base	5.081, std: 0.028	0.690, std: 0.277
only_motors_evolution	5.436, std: 0.156	0.973, std: 0.437

36: Mountain radius

7. Conclusion

Experiment outcomes are summarized:

- Base settings facilitated mountain approach learning and sustained mountain interaction but did not unlock mountain ascent.
- Increasing population improved reliability and sped up solutions discovery but did not unlock mountain ascent.
- Increasing gene count and mutation rate expanded exploration but compounded search complexity, resulting in limited refinement after converging on a solution.
- Stronger motors activity did not unlock mountain ascent but resulted in wasted motor activity due to unreliable motor control.
- Changing the encoding scheme to evolve only motor control genes reduced mountain contact due to inflexible morphology.

Experiment	Last generation elite minimum radius	Improvement in elite minimum radius over generations
base	5.081, std: 0.028	0.690, std: 0.277
gene_5	5.118, std: 0.057	1.090, std: 0.913
mutation_high	5.109, std: 0.042	1.507, std: 0.190
only_motors_evolution	5.436, std: 0.156	0.973, std: 0.437
population_50	5.133, std: 0.044	0.835, std: 0.188
strong_motors	5.195, std: 0.019	0.482, std: 0.484

37: Mountain radius

Conclusively, experiments show positive outcomes but a combination of these approaches is needed to unlock mountain ascent.